Practical -8

Aim:

To Implement of Graph and Searching (DFS and BFS).

Theory:

Depth-First Search (DFS): This algorithm explores as deep as possible along each branch before backtracking. It naturally uses a Stack data structure (implemented via recursion or an explicit list).

Breadth-First Search (BFS): This algorithm explores the graph level-by-level, visiting all immediate neighbors before moving deeper. It uses a Queue (First-In, First-Out) data structure.

Algorithm:

1. Algorithm for DFS

Algorithm: DFS(G, v)

1. Start.
2. Mark vertex v as visited.
3. Display vertex v.
4. For every adjacent vertex u of v:
    - If u is not visited, call DFS(G, u).
5. Stop.

2. Algorithm for BFS

Algorithm: BFS(G, v)

1. Start.

2. Create an empty queue.
3. Mark the starting vertex v as visited.
4. Insert v into the queue.
5. While the queue is not empty:
    - Remove a vertex v from the queue.
    - Display v.
    - For every adjacent vertex u:
        - If u is not visited:
            - Mark u as visited.
            - Insert u into the queue.
6. Stop.

Program Code

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

// DFS function
void DFS(int vertex, vector<vector<int>>& graph,
        vector<bool>& visited, int n) {
```

```
    // Mark current vertex as visited
    visited[vertex] = true;

    // Print current vertex
    cout << vertex << " ";

    // Visit all adjacent vertices
    for (int i = 0; i < n; i++) {
        if (graph[vertex][i] == 1 && !visited[i]) {
            DFS(i, graph, visited, n);
        }
    }
}

// BFS function
void BFS(int start, vector<vector<int>>& graph, int n) {

    vector<bool> visited(n, false);
    queue<int> q;
```

```
// Mark starting vertex as visited
  visited[start] = true;

  // Insert starting vertex into queue
  q.push(start);

  while (!q.empty()) {

     // Remove vertex from queue
     int vertex = q.front();
     q.pop();

     // Print vertex
     cout << vertex << " ";

     // Visit all adjacent vertices
     for (int i = 0; i < n; i++) {

        if (graph[vertex][i] == 1 && !visited[i]) {
           visited[i] = true;
```

```
        q.push(i);
            }
        }
    }
}

int main() {

    int n, start;

    cout << "Enter number of vertices: ";
    cin >> n;

    // Create adjacency matrix
    vector<vector<int>> graph(n, vector<int>(n));

    cout << "Enter adjacency matrix:\n";

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
```

```
cin >> graph[i][j];
    }
 }

 cout << "Enter starting vertex (0 to " << n - 1 << "): ";
 cin >> start;

 // DFS
 vector<bool> visited(n, false);

 cout << "\nDFS Traversal: ";
 DFS(start, graph, visited, n);

 // BFS
 cout << "\nBFS Traversal: ";
 BFS(start, graph, n);

 cout<<"\nSangem vijayendra reddy\n";
 cout<<"92460118171\n";
 cout<<"5-EN18\n";
```

```
  cout << endl;


    return 0;
}
```

Result :